AWS Database Blog

Code conversion patterns to migrate Sybase SQL Anywhere database schemas to Amazon RDS for SQL Server or Microsoft SQL Server on Amazon EC2

by Jitendra Kumar, Bhavani Akundi, and Shankar Padmanabhuni | on 05 MAY 2023 | in Amazon EC2, Amazon RDS, Intermediate (200), RDS for SQL Server, Technical How-to | Permalink | Comments | Share

Resources

Amazon Aurora
Amazon DocumentDB
Amazon DynamoDB
Amazon ElastiCache
Amazon Keyspaces (for Apache Cassandra)
Amazon MemoryDB
Amazon Neptune
Amazon Neptune Analytics
Amazon RDS
Amazon Timestream
AWS Database Migration Service

SAP SQL Anywhere (also known as Sybase SQL Anywhere) is a database which is widely used in Information Technology and Services industry. Migration from SAP SQL Anywhere to Amazon Relational Database Service (RDS) for SQL Server or Microsoft SQL Server on Amazon Elastic Compute Cloud (Amazon EC2) is challenging because as of writing this post, there is no built-in or third-party tool available to perform the code conversion when using SAP SQL Anywhere as a source database.

In this post, we cover the commonly used code conversion patterns to migrate a schema from SAP SQL Anywhere to Amazon RDS for SQL Server or SQL Server on Amazon EC2. Migrating to SQL Server remains a preferred choice for many customers because both Sybase and SQL Server share the same T-SQL dialect as their database programming language and Tabular Data Stream (TDS) as their communication protocol which reduces the overall migration complexity and code conversion timelines.

General considerations

Make sure you have the time zone, character set, and collation settings in line with the source database while creating the target DB instances on AWS. You can refer Collations and character sets for Microsoft SQL Server for RDS for SQL Server or **configuring SQL Server** section under Best practices for deploying Microsoft SQL Server on Amazon EC2.

Data types

Heterogeneous migration provides the opportunity to streamline the data types. The following table lists the data types supported in SAP SQL Anywhere and the equivalent data types in SQL Server.

SAP SQL Anywhere	Microsoft SQL Server	Description
BIGDATETIME	DATETIME2(6)	Date and time with fraction
BIGINT	BIGINT	64-bit integer
BIGTIME	TIME(6)	Time (hour, minute, second, and fraction)
BINARY(*n*)	BINARY(*n*)	Fixed-length binary string
BIT	BIT	0 or 1; NULL is not allowed
CHAR(*n*), CHARACTER(*n*)	CHAR(*n*), CHARACTER(*n*)	Fixed-length string
DATE	DATE	Date (year, month, and day)
DATETIME	DATETIME2(6)	Date and time with fraction
DECIMAL(*p,s*), DEC(*p,s*)	DECIMAL(*p,s*), DEC(*p,s*)	Fixed-point number
DOUBLE PRECISION	FLOAT	Double-precision floating-point number
FLOAT(*p*)	FLOAT	Floating-point number
IMAGE	VARBINARY(max)	Variable-length binary data, ⇐ 2G
INT, INTEGER	INT, INTEGER	32-bit integer
MONEY	MONEY	64-bit currency amount
NCHAR(*n*)	NCHAR(*n*)	Fixed-length national character string
NUMERIC(*p,s*)	NUMERIC(*p,s*)	Fixed-point number
NVARCHAR(*n*)	NVARCHAR(*n*)	Variable-length national character string
REAL	REAL	Single-precision floating-point number
SMALLDATETIME	SMALLDATETIME	Date and time
SMALLINT	SMALLINT	16-bit integer
SMALLMONEY	SMALLMONEY	32-bit currency amount
TEXT	VARCHAR(max)	Variable-length character data, ⇐ 2G
TIME	TIME(6)	Time (hour, minute, second, and fraction)
TINYINT	TINYINT	8-bit unsigned integer, 0 to 255
UNICHAR(*n*)	NCHAR(*n*)	Fixed-length Unicode string
UNITEXT	NVARCHAR(max)	Variable-length Unicode data, ⇐ 1G
UNIVARCHAR(*n*)	NVARCHAR(*n*)	Variable-length Unicode string
UNSIGNED BIGINT	NUMERIC(20)	64-bit unsigned integer
UNSIGNED INT	NUMERIC(10)	32-bit unsigned integer
UNSIGNED SMALLINT	NUMERIC(5)	16-bit unsigned integer
VARBINARY(*n*)	VARBINARY(*n*)	Variable-length binary string
VARCHAR(*n*)	VARCHAR(*n*)	Variable-length string

Functions

The following table lists the built-in functions supported in SAP SQL Anywhere and the equivalent functions in SQL Server.

SAP SQL Anywhere	Microsoft SQL Server	Description
ABS(*num*)	ABS(*num*)	Get the absolute value
ACOS(*num*)	ACOS(*num*)	Get the arc cosine
ASCII(*str*)	ASCII(*str*)	Get ASCII code of left-most char
ASIN(*num*)	ASIN(*num*)	Get the arc sine
ATAN(*num*)	ATAN(*num*)	Get the arc tangent
CEILING(*num*)	CEILING(*num*)	Get the smallest following integer
CHAR(*num*)	CHAR(*num*)	Get character from ASCII code
CHAR_LENGTH(*string*)	LEN(string)	Get length of string in characters. It supports both Unicode and ASCII however the returned length varies based on the default character set applied.
CHARINDEX(*substr, str*)	CHARINDEX(*substr, str*)	Get position of substr
COALESCE(exp1, exp2, …)	COALESCE(exp1, exp2, …)	Return first non-NULL expression
COMPARE(*str, str2*)	User-defined function	Compare two strings
CONVERT(datatype, exp)	CONVERT(datatype, exp)	Convert to another datatype
CURRENT DATE	CONVERT(DATE, GETDATE())	Get the current date
CURRENT TIME	CONVERT(TIME, GETDATE())	Get the current time
DATALENGTH(*exp*)	DATALENGTH(*exp*)	Get length of exp in bytes
DATEADD(unit, num, date)	DATEADD(unit, num, date)	Add an interval to datetime
DATEDIFF(unit, start, end)	DATEDIFF(unit, start, end)	Get datetime difference in units
DATENAME(datepart, date)	DATENAME(datepart, date)	Get datepart as string
DATEPART(datepart, date)	DATEPART(datepart, date)	Get datepart as integer
DAY(datetime)	DAY(datetime)	Extract day from datetime
DB_ID(['database_name'])	DB_ID(['database_name'])	Get ID number of database
DB_INSTANCEID()	@@SERVICENAME	Get ID number of instance
DB_NAME(['database_id'])	DB_NAME(['database_id'])	Get the name of database
DEGREES(*num*)	DEGREES(*num*)	Convert radians to degrees
EXP(*n*)	EXP(*n*)	Raise to the n th power
FLOOR(*num*)	FLOOR(*num*)	Get the largest preceding integer
GETDATE()	GETDATE()	Get the current date and time
GETUTCDATE()	GETUTCDATE()	Get the current UTC datetime
HEXTOBIGINT(*exp*)	CONVERT(BIGINT, *exp*)	Convert hexadecimal to bigint
HEXTOINT(*exp*)	CONVERT(INT, *exp*)	Convert hexadecimal to integer
INDEX_COL(obj, id, key)	INDEX_COL(obj, id, key)	Get indexed column name
INDEX_COLORDER	User-defined function	Get column order
INDEX_NAME	User-defined function	Get index name
@@SERVERNAME	@@SERVERNAME	Get name of instance
ISDATE(*string*)	ISDATE(*string*)	Check for a valid datetime
ISNULL(exp, replace)	ISNULL(exp, replace)	Replace NULL
ISNUMERIC(*string*)	ISNUMERIC(*string*)	Check for a valid numeric
LEFT(string, n)	LEFT(string, n)	Get nth leftmost characters
LEN(string)	LEN(string)	Get length of string in characters. It supports both Unicode and ASCII however the returned length varies based on the default character set applied.
LOG(numeric)	LOG(numeric)	Get natural logarithm
LOWER(*string*)	LOWER(*string*)	Lowercase string
LTRIM(*string*)	LTRIM(*string*)	Remove leading spaces
MONTH(*date*)	MONTH(*date*)	Extract month from date
NULLIF(exp1, exp2)	NULLIF(exp1, exp2)	Return NULL if exp1=exp2
POWER(value, n)	POWER(value, n)	Raise value to the nth power
RAND([*integer*])	RAND([*integer*])	Get random float value in (0, 1)
REPLICATE(*string, n*)	REPLICATE(*string, n*)	Repeat string n times
REVERSE(*string*)	REVERSE(*string*)	Get reverse string
RIGHT(string, n)	RIGHT(string, n)	Get n rightmost characters
ROUND(num, integer)	ROUND(num, integer)	Get rounded value
RTRIM(*string*)	RTRIM(*string*)	Remove trailing spaces
SIGN(*exp*)	SIGN(*exp*)	Get sign of exp
SIN(*num*)	SIN(*num*)	Get sine
SOUNDEX(*string*)	SOUNDEX(*string*)	Get 4-character sound code
SPACE(*integer*)	SPACE(*integer*)	Get string of spaces
SQUARE(*exp*)	SQUARE(*exp*)	Get square
SQRT(*num*)	SQRT(*num*)	Get square root
STR_REPLACE(*s, sub, r*)	REPLACE(*s, sub, r*)	Replace substring
STUFF(exp, start, len, rep)	STUFF(exp, start, len, rep)	Replace characters in string
SUBSTRING(exp, pos, len)	SUBSTRING(exp, pos, len)	Get a substring of exp
SUSER_ID(*exp*)	SUSER_ID(*exp*)	Get the server user's ID
TEXTVALID(exp, pointer)	TEXTVALID(exp, pointer)	Check validity of a pointer
TO_UNICHAR(*int*)	CONVERT(NVARCHAR, *int*)	Convert int to Unicode character
TSEQUAL(value1, value2)	value1 = value2	Compare two timestamp values
UPPER(*string*)	UPPER(string)	Uppercase string
USER	SYSTEM_USER	Get the current user
USER_ID([*user_name*])	USER_ID([*user_name*])	Get user ID
USER_NAME([*user_id*])	USER_NAME([*user_id*])	Get user name
YEAR(*date*)	YEAR(*date*)	Extract year from date

Now let's look at some of the commonly used code conversion patterns.

Pattern 1: Get the length of a string

To return the number of characters in the specified string.

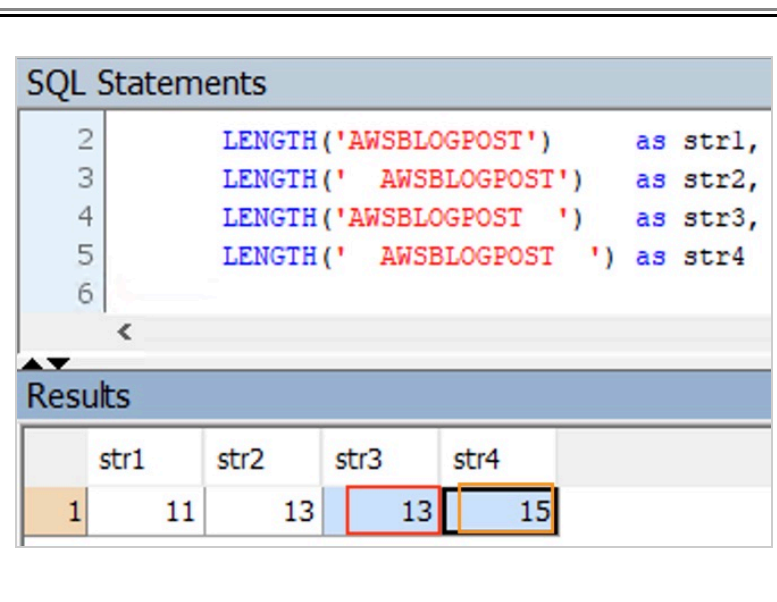
Sybase	SQL Server
LENGTH function is used, but it doesn't exclude trailing spaces	LEN function is used, but it excludes trailing spaces

Sybase

```
SELECT
        LENGTH('AWSBLOGPOST')      as str1,
        LENGTH('   AWSBLOGPOST')   as str2,
        LENGTH('AWSBLOGPOST   ')   as str3,
        LENGTH('   AWSBLOGPOST   ') as str4
```

SQL Server

```
SELECT
        LEN('AWSBLOGPOST')             as str1,
        LEN('   AWSBLOGPOST')          as str2,
        LEN('AWSBLOGPOST   ')          as str3,
        LEN('   AWSBLOGPOST   ')       as str4
```



SQL Statements

```
2         LENGTH('AWSBLOGPOST')       as str1,
3         LENGTH('   AWSBLOGPOST')    as str2,
4         LENGTH('AWSBLOGPOST   ')    as str3,
5         LENGTH('   AWSBLOGPOST   ') as str4
6
```

Results

	str1	str2	str3	str4
1	11	13	13	15

Note that the difference in the output of **str3** and **str4** are because SQL Server excludes trailing spaces but Sybase does not.

Pattern 2: Find the position of one string within another string

To search one character expression inside a second character expression, and return the starting position of the first expression if found.

Sybase	SQL Server
LOCATE function is used	CHARINDEX function is used

Sybase

```
BEGIN
    DECLARE @document VARCHAR(64);

    SELECT @document = 'Reflectors are vital safety' +
                       ' components of your bicycle.';

    SELECT LOCATE(@document, 'vital');

    SELECT LOCATE(@document, 'vital',17);
END
```

SQL Server

```
BEGIN
    DECLARE @document VARCHAR(64);

    SELECT @document = 'Reflectors are vital safety' +
                       ' components of your bicycle.';

    SELECT CHARINDEX('vital', @document);

    SELECT CHARINDEX('vital', @document, 17);
END
```

Note that in Sybase the string to be searched is the second parameter while in SQL Server, it is the first parameter.

Pattern 3: Return a substring of a string

To extract some character from a specified string.

Sybase	SQL Server
SUBSTRING or SUBSTR function are used	SUBSTRING is used

Sybase

```
SELECT SUBSTRING('AWSBLOGPOST',4)
```

SQL Server

```
SELECT SUBSTRING('AWSBLOGPOST',4,
           LEN('AWSBLOGPOST'))
```

Note that in SQL Server, third parameter (length) is mandatory.

Pattern 4: DATALENGTH function

To return the length, in bytes, of the underlying storage for the result of an expression. For a NULL value return NULL.

Sybase	SQL Server
DATALENGTH function is used	DATALENGTH function is used

Sybase

```
CREATE TABLE DEMO(
                  id INTEGER,
                  ename CHAR(10),
                  doj DATE,
                  address VARCHAR(255)
                 )

INSERT INTO DEMO VALUES
(100, 'Jitendra', '2004-01-01', '99, XYZ Street, India - 999999')
INSERT INTO DEMO VALUES
(1000, 'Bhavani', '2005-12-31', '101, ABC Street, India - 123456 ')

SELECT
    DATALENGTH(id) as dl_id,
    DATALENGTH(ename) as dl_ename,
    DATALENGTH(doj) as dl_doj,
    DATALENGTH(address) as dl_address
FROM DEMO
```

SQL Server

```
CREATE TABLE DEMO(
                  id INTEGER,
                  ename CHAR(10),
                  doj DATE,
                  address VARCHAR(255)
                 )

INSERT INTO DEMO VALUES
(100, 'Jitendra', '2004-01-01', '99, XYZ Street, India - 999999')
INSERT INTO DEMO VALUES
(1000, 'Bhavani', '2005-12-31', '101, ABC Street, India - 123456 ')

SELECT
    DATALENGTH(id) as dl_id,
    DATALENGTH(ename) as dl_ename,
    DATALENGTH(doj) as dl_doj,
    DATALENGTH(address) as dl_address
FROM DEMO
```

Note the differences for the DATE and CHAR data type columns between Sybase and SQL Server.

For the CHAR data type, SQL Server returns a fixed length, whereas Sybase returns the actual length. To return the number of characters in a string expression, use the LEN function.

For the DATE data type, SQL Server returns a fixed length of 3 bytes, whereas Sybase returns 4 bytes.

Pattern 5: SELECT INTO hostvar-list clause

Let's cover the examples related to Transact-SQL (T-SQL). Here we want to highlight how T-SQL differs between Sybase and SQL Server. The reason is that SQL Server strictly follows the ANSI standard, whereas Sybase is a bit lenient.

Sybase

```
BEGIN
    DECLARE @gid INTEGER
    DECLARE @name VARCHAR(100)

    SELECT
        gid, name INTO @gid, @name
    FROM sysusers WHERE uid = 0;

    SELECT @gid, @name
END
```

SQL Server

```
BEGIN
    DECLARE @gid INTEGER
    DECLARE @name VARCHAR(100)

    SELECT
        @gid = gid, @name = name
    FROM sysusers
    WHERE uid = 4;

    SELECT @gid, @name
END
```

Pattern 6: SELECT INTO TABLE table-name clause

The SELECT…INTO clause always creates a table and populates it with the results of the query. The behaviour is same in both Sybase and SQL Server.

Sybase

```
SELECT id, ename INTO DEMO_NEW FROM DEMO

SELECT * FROM DEMO_NEW
```

SQL Server

```
SELECT id, ename INTO DEMO_NEW FROM DEMO

SELECT * FROM DEMO_NEW
```

Pattern 7: PRINT statement

The PRINT statement is used to return a message to the client, or display a message in the database server messages window.

Sybase

```
PRINT ' Timestamp = %1!', GETDATE()
```

SQL Server

```
PRINT CONCAT(' Timestamp = ' , GETDATE())
```

Note that in Sybase, **%1!** in a print statement retrieves the first assigned value, whereas in SQL Server, you use a concatenate function to display dynamic values.

Pattern 8: CASE statement

The CASE statement is a control statement that lets you choose a list of SQL statements to execute based on the value of an expression.

Sybase

```
begin
    declare @lAmountValue decimal(12,2)
    declare @lComponent   char(1) = 'T'
    declare @lPcTicket    decimal(12,2) = 111.99
    declare @lPcTax       decimal(12,2) = 222.99
    declare @lPcOther1    decimal(12,2) = 333.99

    BEGIN
        CASE @lComponent
          WHEN 'T'    THEN  SELECT @lAmountValue = @lPcTicket
          WHEN 'X'    THEN  SELECT @lAmountValue = @lPcTax
          WHEN '1'    THEN  SELECT @lAmountValue = @lPcOther1
        END
    END

SELECT
     @lComponent,@lPcTicket,@lPcTax,
     @lPcOther1,@lAmountValue
end
```

SQL Server

```
begin
    declare @lAmountValue decimal(12,2)
    declare @lComponent char(1) = 'T'
    declare @lPcTicket decimal(12,2) = 111.99
    declare @lPcTax decimal(12,2)    = 222.99
    declare @lPcOther1 decimal(12,2) = 333.99

    BEGIN
        SELECT @lAmountValue =
            CASE @lComponent
                WHEN 'T'      THEN              @lPcTicket
                WHEN 'X'      THEN              @lPcTax
                WHEN '1'      THEN              @lPcOther1
            END
    END

    SELECT @lComponent,@lPcTicket,
                @lPcTax,@lPcOther1,@lAmountValue
end
```

Note that the value of **@lAmountValue** variable changes based on the value of **@lComponent** variable.

Pattern 9: Shared global temporary tables

Sybase supports both shared and non-shared global temporary tables. In SQL Server, there is no concept of shared and non-shared global temporary tables. SQL Server has global temporary tables, and their definitions and content are shared across all the sessions.

Sybase	SQL Server
CREATE GLOBAL TEMPORARY TABLE statement creates a global temporary table in the database. The rows in the table may or may not be shared depending on the SHARE BY ALL clause.	SELECT – INTO clause creates a new table and inserts the resulting rows from the query into it

Sybase

```
CREATE TABLE DEMO(
                   id INTEGER,
                   ename CHAR(10),
                   doj DATE,
                   address VARCHAR(255)
                  );

CREATE GLOBAL TEMPORARY TABLE DEMO_SHARED_TEMP
                 (
                   EmpID,
                   EmpName,DOJ
                 )
AS (SELECT id, ename, doj FROM DEMO)
ON COMMIT PRESERVE ROWS
SHARE BY ALL
WITH NO DATA;

SELECT * FROM DEMO_SHARED_TEMP;
```

SQL Server

```
CREATE TABLE DEMO(
            id INTEGER,
            ename CHAR(10),
            doj DATE,
            address VARCHAR(255)
           );

SELECT
      id           as EmpID ,
      ename        as EmpName ,
      doj          INTO DEMO_SHARED_TEMP
FROM DEMO;

SELECT * FROM DEMO_SHARED_TEMP;
```

Note that you prefix the local temporary table names with single hash sign (#table_name), and prefix the global temporary table names with a double hash sign (##table_name).

Consider creating a permanent table DEMO_SHARED_TEPM instead of a global temporary table in SQL Server.

Pattern 10: ERROR handling

Error handling is the process of controlling unwanted or unexpected events when a T-SQL code is ran. It provides the ability to return human readable error messages instead of returning a system defined message.

Sybase	SQL Server
The database server sets a SQLSTATE and SQLCODE for each SQL statement it runs	@@ERROR returns the error number for the last Transact-SQL statement run

Sybase

```
begin
    DECLARE @oSqlCode        integer
    DECLARE @oSqlState       char( 5)
    DECLARE @oErrorMsg       varchar(254)
    DECLARE @empName         varchar(254)

    SELECT
       @empName = ename
    FROM demo
    WHERE id = 1000

    SELECT @oSqlCode  = SQLCODE
         , @oSqlState = SQLSTATE

    IF @oSqlState = '02000'  -- record not found
      BEGIN
        SELECT @returnCode       = -1
        SELECT @oErrorMsg = 'Employee ID not found.'
      END
```

SQL Server

```
begin
    DECLARE @oSqlCode     integer
    DECLARE @oSqlState    char( 5)
    DECLARE @oErrorMsg    varchar(254)
    DECLARE @RowCount     int
    DECLARE @empName      varchar(254)

    SELECT
       @empName = ename
    FROM demo
    WHERE id = 1000

    SELECT @oSqlCode    = @@ERROR
         , @RowCount    = @@ROWCOUNT

    IF @oSqlCode = 0
    BEGIN
        IF @RowCount = 0   -- record not found
        BEGIN
```

Sybase	SQL Server
Run the preceding code with WHERE id = 9999 in Sybase	Run the preceding code with WHERE id = 9999 in SQL Server

Conclusion

In this post, we explained the commonly used code conversion patterns and best practices for developers migrating SAP SQL Anywhere to Amazon RDS for SQL Server or SQL Server on Amazon EC2. We also highlighted some decisions you have to make while creating a SQL Server database that can impact your database performance. Keeping these performance aspects in mind during the conversion can help avoid performance issues later on during migration.

If you have any questions or comments about this post, share your thoughts in the comments section.

About the Authors

Jitendra Kumar is a lead database migration specialist with AWS Professional Services. He works closely with customers to help migrate and modernize their databases and applications to AWS.

Bhavani Akundi is a lead database consultant at AWS Professional Services. She has vast experience working with Microsoft Technologies with a specialization in SQL Server.

Shankar Padmanabhuni is a team lead for database consultants with Professional Services at AWS. His team helps AWS customers modernize, architect, and build highly scalable cloud-native applications on AWS.

Like Share

1 Comment

Log in to comment

Sort by: **Top**

Learn
What Is AWS?
What Is Cloud Computing?
What Is Agentic AI?
Cloud Computing Concepts Hub
AWS Cloud Security
What's New
Blogs
Press Releases

Resources
Getting Started
Training
AWS Trust Center
AWS Solutions Library
Architecture Center
Product and Technical FAQs
Analyst Reports
AWS Partners

Developers
Builder Center
SDKs & Tools
.NET on AWS
Python on AWS
Java on AWS
PHP on AWS
JavaScript on AWS

Help
Contact Us
File a Support Ticket
AWS re:Post
Knowledge Center
AWS Support Overview
Get Expert Help
AWS Accessibility
Legal

Back to top

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.
Privacy Site terms Cookie Preferences
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.